ITA0620 - MACHINE LEARNING

Lab Exercises Report

Field	Details
Name	Pranayaa S
Register Number	192524110
Course Code	ITA0620
Course Name	Machine Learning

Index

S.No.	Experiment	Program File
1	FIND-S Algorithm	exp1_find_s.py
2	Candidate-Elimination Algorithm	exp2_candidate_elimination.py
3	Decision Tree using ID3 Algorithm	exp3_id3.py

EXPERIMENT 1: FIND-S ALGORITHM

Aim

To implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Algorithm

1. Initialize hypothesis h to the most specific hypothesis in H, i.e. h = <0, 0, 0, 0, 0, 0>.
2. For each training instance x: if x is a negative example, ignore it and continue.
3. If x is a positive example, then for each attribute constraint ai in h: if ai is satisfied by x, do nothing; otherwise replace ai with the next more general constraint ('?') that is satisfied by x.
4. After all training examples are processed, output hypothesis h.

Dataset (enjoysport.csv)

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Program (exp1_find_s.py)

```python
# Experiment 1: FIND-S Algorithm
# Name: Pranayaa S | Reg No: 192524110 | ITA0620 - Machine Learning
import csv

def find_s(filename):
    with open(filename) as f:
        reader = csv.reader(f)
        header = next(reader)
        data = [row for row in reader]

    print("Attributes:", header[:-1])
    print("Training data:")
    for row in data:
        print("  ", row)

    hypothesis = ['0'] * (len(header) - 1)   # most specific hypothesis
    print("\nInitial hypothesis:", hypothesis)

    step = 0
    for row in data:
        step += 1
        if row[-1].strip().lower() == 'yes':          # positive examples only
            for i in range(len(hypothesis)):
                if hypothesis[i] == '0':
                    hypothesis[i] = row[i]
                elif hypothesis[i] != row[i]:
                    hypothesis[i] = '?'
            print(f"After example {step} (positive): {hypothesis}")
        else:
            print(f"After example {step} (negative, ignored): {hypothesis}")

    print("\nMaximally specific hypothesis:", hypothesis)
    return hypothesis

if __name__ == "__main__":
    find_s("enjoysport.csv")
```

Output

```
Windows PowerShell - exp1_find_s.py
PS C:\Users\Pranayaa\ML_Lab> python exp1_find_s.py
Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Training data:
   ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
   ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
   ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
   ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Initial hypothesis: ['0', '0', '0', '0', '0', '0']
After example 1 (positive): ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
After example 2 (positive): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After example 3 (negative, ignored): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After example 4 (positive): ['Sunny', 'Warm', '?', 'Strong', '?', '?']

Maximally specific hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
PS C:\Users\Pranayaa\ML_Lab>
```

Result

The FIND-S algorithm was implemented successfully. The maximally specific hypothesis obtained for the EnjoySport dataset is <Sunny, Warm, ?, Strong, ?, ?>.

EXPERIMENT 2: CANDIDATE-ELIMINATION ALGORITHM

Aim

For a given set of training data examples stored in a .CSV file, to implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Algorithm

1. Initialize G to the set of maximally general hypotheses <?, ?, ?, ?, ?, ?> and S to the set of maximally specific hypotheses.
2. For each training example d: if d is a positive example, remove from G any hypothesis inconsistent with d, and minimally generalize S so that it is consistent with d.
3. If d is a negative example, remove from S any hypothesis inconsistent with d, and replace each inconsistent member of G with its minimal specializations that are consistent with d and more general than S.
4. Remove from G any hypothesis that is less general than another hypothesis in G.
5. After all examples are processed, output the specific boundary S and general boundary G; together they describe the version space.

Dataset (enjoysport.csv)

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Program (exp2_candidate_elimination.py)

```
# Experiment 2: Candidate-Elimination Algorithm
# Name: Pranayaa S | Reg No: 192524110 | ITA0620 - Machine Learning
import csv

def load_data(filename):
    with open(filename) as f:
        reader = csv.reader(f)
        header = next(reader)
        data = [row for row in reader]
    concepts = [row[:-1] for row in data]
    target = [row[-1] for row in data]
    return header, concepts, target

def candidate_elimination(concepts, target):
    n = len(concepts[0])
    S = concepts[0].copy() if target[0].lower() == 'yes' else ['0'] * n
    G = [['?'] * n for _ in range(n)]
    print("Initial S:", S)
    print("Initial G:", [['?'] * n])

    for i, x in enumerate(concepts):
        if target[i].lower() == 'yes':          # positive example
```

```
            for j in range(n):
                if S[j] != x[j]:
                    S[j] = '?'
                    G[j][j] = '?'
        else:                                   # negative example
            for j in range(n):
                if S[j] != x[j]:
                    G[j][j] = S[j]
                else:
                    G[j][j] = '?'
        print(f"\nStep {i+1} ({target[i]}): {x}")
        print("  S =", S)
        print("  G =", [g for g in G if g != ['?'] * n] or [['?'] * n])

    G = [g for g in G if g != ['?'] * n]
    return S, G

if __name__ == "__main__":
    header, concepts, target = load_data("enjoysport.csv")
    print("Attributes:", header[:-1])
    S, G = candidate_elimination(concepts, target)
    print("\nFinal Specific boundary (S):", S)
    print("Final General boundary (G):", G)
```

Output

```
Windows PowerShell - exp2_candidate_elimination.py
PS C:\Users\Pranayaa\ML_Lab> python exp2_candidate_elimination.py
Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Initial S: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
Initial G: [['?', '?', '?', '?', '?', '?']]

Step 1 (Yes): ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
  S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
  G = [['?', '?', '?', '?', '?', '?']]

Step 2 (Yes): ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same']
  S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
  G = [['?', '?', '?', '?', '?', '?']]

Step 3 (No): ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change']
  S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]

Step 4 (Yes): ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change']
  S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]

Final Specific boundary (S): ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final General boundary (G): [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
PS C:\Users\Pranayaa\ML_Lab>
```

Result

The Candidate-Elimination algorithm was implemented successfully. Final specific boundary S = <Sunny, Warm, ?, Strong, ?, ?> and general boundary G = {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}.

EXPERIMENT 3: DECISION TREE USING ID3 ALGORITHM

Aim

To demonstrate the working of the decision tree based ID3 algorithm using an appropriate dataset for building the decision tree and to apply this knowledge to classify a new sample.

Algorithm

1. Compute the entropy of the target attribute for the whole dataset: Entropy(S) = - Σ p_i log2(p_i).
2. For each attribute A, compute the information gain: Gain(S, A) = Entropy(S) - Σ (|Sv| / |S|) × Entropy(Sv).
3. Select the attribute with the highest information gain as the decision node.
4. Split the dataset on each value of the selected attribute and create a branch for each value.
5. Repeat steps 1-4 recursively on each subset using the remaining attributes, until all examples in a subset share the same class (leaf node) or no attributes remain (majority class).
6. To classify a new sample, traverse the tree from the root following the branches that match the sample's attribute values until a leaf is reached.

Dataset (playtennis.csv)

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Program (exp3_id3.py)

```
# Experiment 3: Decision Tree using ID3 Algorithm
# Name: Pranayaa S | Reg No: 192524110 | ITA0620 - Machine Learning
```

```
import csv
import math
from collections import Counter

def load_data(filename):
    with open(filename) as f:
        reader = csv.reader(f)
        header = next(reader)
        data = [row for row in reader]
    return header, data

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, col):
    total = len(rows)
    values = set(r[col] for r in rows)
    remainder = 0
    for v in values:
        subset = [r for r in rows if r[col] == v]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs, header):
    labels = [r[-1] for r in rows]
    if labels.count(labels[0]) == len(labels):
        return labels[0]
    if not attrs:
        return Counter(labels).most_common(1)[0][0]
    gains = {a: info_gain(rows, a) for a in attrs}
    best = max(gains, key=gains.get)
    tree = {header[best]: {}}
    remaining = [a for a in attrs if a != best]
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        tree[header[best]][v] = id3(subset, remaining, header)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + tree)
        return
    for attr, branches in tree.items():
        for value, subtree in branches.items():
            print(f"{indent}{attr} = {value}")
            print_tree(subtree, indent + "    ")

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    return classify(tree[attr][sample[attr]], sample)

if __name__ == "__main__":
    header, data = load_data("playtennis.csv")
    attrs = list(range(len(header) - 1))
    print("Entropy of full dataset: %.4f" % entropy(data))
    print("Information gain of each attribute:")
    for a in attrs:
        print("  %-12s %.4f" % (header[a], info_gain(data, a)))

    tree = id3(data, attrs, header)
    print("\nDecision Tree:")
    print_tree(tree)
```

```
    sample = {"Outlook": "Sunny", "Temperature": "Cool",
              "Humidity": "High", "Wind": "Strong"}
    print("\nNew sample:", sample)
    print("Predicted PlayTennis:", classify(tree, sample))
```

Output

```
Windows PowerShell - exp3_id3.py
PS C:\Users\Pranayaa\ML_Lab> python exp3_id3.py
Entropy of full dataset: 0.9403
Information gain of each attribute:
  Outlook      0.2467
  Temperature  0.0292
  Humidity     0.1518
  Wind         0.0481

Decision Tree:
Outlook = Overcast
    -> Yes
Outlook = Rain
    Wind = Strong
        -> No
    Wind = Weak
        -> Yes
Outlook = Sunny
    Humidity = High
        -> No
    Humidity = Normal
        -> Yes

New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Predicted PlayTennis: No
PS C:\Users\Pranayaa\ML_Lab>
```

Result

The ID3 algorithm was implemented successfully. 'Outlook' was selected as the root node (highest gain 0.2467), and the new sample <Sunny, Cool, High, Strong> was classified as PlayTennis = No.